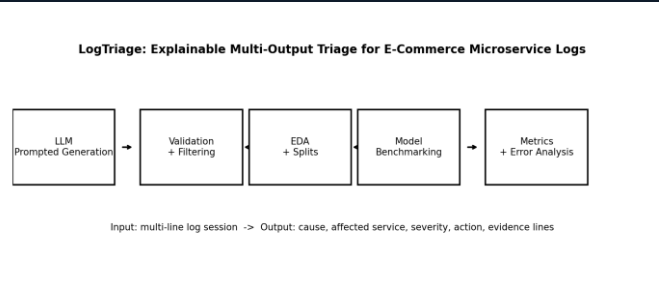


Motivating use case

E-commerce checkout incident



Checkout failures affect users, orders, and revenue.
A single failure creates logs across gateway, auth, cart, payment, inventory, order, database, and queues.
Operators must quickly decide: what failed, where, how severe, what action, and what evidence.

Why it is challenging

Logs are noisy and distributed across services.
Symptom lines often hide the root cause. Actions must be specific and consistent with the failure.
Evidence must be traceable to log line numbers.

Today

manual log reading + keyword search + runbooks

Project idea

Turn noisy logs into structured operator triage:



Scope: a fictional ShopFlow e-commerce microservice platform. The method can later adapt to other domains.

Project task description

Formal problem statement

Given a numbered multi-line log session from an e-commerce microservice system, predict a structured triage JSON with the root failure, affected service, severity, action, and evidence lines.

Input log session

```
[1] gateway INFO POST /checkout request_id=req_912
[2] checkout INFO validating cart_id=cart_771
[3] payment-service WARN provider_latency=4900ms
[4] payment-service ERROR payment gateway timeout after 5000ms
[5] order-service ERROR order creation failed: payment authorization missing
[6] gateway ERROR status=503 route=/checkout
```

Key novelty

Not only abnormal/normal. LogTriage produces operator-ready structured decisions with traceable evidence.

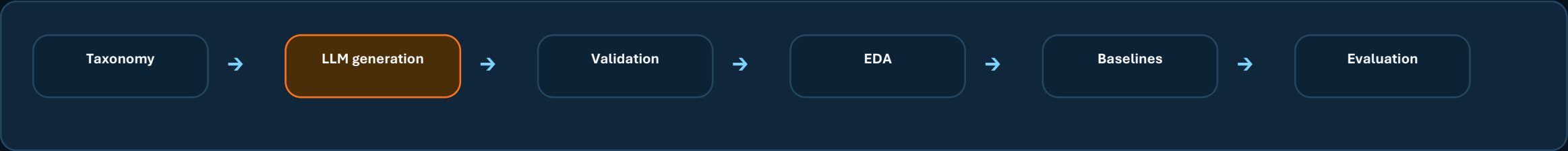
Structured output schema

```
{
  "failure_cause": "payment_gateway_timeout",
  "affected_service": "payment-service",
  "severity": "high",
  "recommended_action": "check_payment_gateway_and_retry",
  "evidence_lines": [3, 4, 6]
}
```

Fixed taxonomies make the task measurable

failure_cause: 12 classes such as payment timeout, database query failure, authentication failure
affected_service: gateway, auth, payment, inventory, database, queue, etc.
recommended_action: retry, check credentials, scale, rollback, inspect dependency, open incident

Models and methods



Generation + validation

LLM creates log sessions from controlled attributes.
Validator checks JSON, allowed labels, evidence-line references, duplicate sessions, and class balance.
Manual spot-checking planned for a gold subset.

Model comparison plan

Rule baseline	keywords/runbook rules
TF-IDF + LogReg/SVM	classical NLP baseline
Zero/Few-shot LLM	prompt-only benchmark
DistilBERT/RoBERTa	fine-tuned extension

Fine-tuning approach: train classifiers for cause, service, severity, and action; evaluate evidence lines separately.

Quality adjustments

Balance failure classes and severity levels.
Vary noise level, length, wording, service path, and log format.
Add misleading symptom lines to reduce keyword shortcuts.
Use error analysis to refine generation prompts.

Data specification and generation

Data schema

Each example contains:

session_id
numbered logs[]
failure_cause
affected_service
severity
recommended_action
evidence_lines[]
noise_level + metadata

5,000

target examples

70/15/15

split

12

failure types

0

schema issues

Controlled attributes

failure cause

affected service

severity

recommended action

noise level

log length

log style

service path

Why controlled generation?

It improves diversity, balance, and reproducibility compared with asking the LLM to generate examples randomly.

Generation prompt flow

Attributes



Prompt template



LLM output



Validation + split

Generated examples are labeled by construction, then filtered for consistency.

Metrics and KPIs

End-to-end KPIs

Accuracy and macro-F1 for each classification output.
Evidence-line precision, recall, and F1.
Full JSON exact match: all fields correct together.
Valid JSON rate for LLM outputs.

Ground truth

Synthetic labels from controlled generation + schema validation + manual review subset.

Metric table

Output	Metric
cause	accuracy, macro-F1
service	accuracy, macro-F1
severity	accuracy, macro-F1
action	accuracy, macro-F1
evidence	precision, recall, F1
full JSON	exact match

Quality measurements

Data quality: class balance, duplicate rate, length distribution, noise distribution.
Model quality: benchmark table + confusion matrix + error analysis.
Explainability: evidence-line scores and examples of correct/wrong evidence.
Robustness: compare performance by noise level and failure type.

Expected final result

A complete pipeline that proves structured log triage is measurable and feasible on controlled synthetic logs.